PROJET DE MACHINE LEARNING · M2 DATASCALE

DeiT : comprendre, puis implémenter

Training data-efficient image transformers & distillation through attention

Touvron et al., ICML 2021 · Implémentation PyTorch sur CIFAR-10

1 · LE PROBLÈME

Les Transformers de vision sont puissants, mais gourmands

Avant DeiT

Le ViT demande des centaines de millions d’images et de grosses infrastructures pour battre les convnets.

Idée de DeiT

Entraîner un ViT sur ImageNet seul, sans données externes, grâce à une bonne recette et à la distillation.

Notre projet

Une version réduite, DeiT-Tiny distillé, sur CIFAR-10 et sur CPU, pour comprendre chaque pièce du papier.

Touvron et al., ICML 2021

2 · ARCHITECTURE

L’image devient une séquence de tokens

ENTRÉE

32×32×3

Image CIFAR-10

PATCHS

64 patchs

4×4, soit 48 valeurs chacun, projetés en 192 dimensions par une Conv2d

TOKENS

66 tokens

64 patchs + CLS + DIST, plus des positions apprises

TRANSFORMER

12 blocs

La forme reste (B, 66, 192) du début à la fin

Classes : **PatchEmbedding**, **TokenEmbedding**, **TransformerEncoder**. Le Transformer ne connaît pas l’espace 2D, c’est l’embedding de position qui lui donne l’ordre.

DeiT-Tiny : 192 dimensions, 3 têtes, 12 couches

2 · ARCHITECTURE

Un bloc : attention, MLP et connexions résiduelles

```
x = x + drop_path( attn( norm1(x) ) )
x = x + drop_path( mlp( norm2(x) ) )
```

Attention

softmax(QK^T/√d)·V : chaque token choisit les tokens qu’il lit.

MLP

192 → 768 → 192 avec GELU, appliqué à chaque token séparément.

LayerNorm

Normalise les 192 valeurs de chaque token, indépendamment du batch.

Résiduelle

Le bloc apprend une correction, et le gradient passe par l’addition.

Classes : Attention, MLP, TransformerBlock

2 · ARCHITECTURE

DropPath : sauter des blocs pour régulariser

Le principe

Pendant l’entraînement, la branche d’un bloc est supprimée au hasard pour chaque image. Seule la connexion résiduelle reste.

On divise par keep_prob pour garder la même moyenne en sortie. À l’évaluation, rien n’est supprimé.

Un taux croissant

Le taux passe de 0 (bloc 1) à 0.1 (bloc 12), de façon linéaire.

Les premiers blocs construisent ce dont tous les suivants dépendent. Les derniers ne font qu’affiner, donc les sauter coûte moins cher.

Classes : DropPath, TransformerEncoder (torch.linspace)

3 · DISTILLATION

Un token de distillation à côté du token de classe

Token CLS

Sa tête (head) apprend le vrai label de l’image.

Token DIST

Sa tête (head_dist) apprend la décision du professeur, un convnet figé.

Les deux tokens traversent les mêmes 12 blocs et se parlent dans l’attention. Ils partent d’embeddings différents et reçoivent des gradients de cibles différentes. Dans le papier, leur similarité cosinus passe de **0.06** en entrée à **0.93** en sortie.

Classe : DeiTTinyDistilled.forward renvoie (cls_logits, dist_logits)

3 · DISTILLATION

Deux pertes possibles : dure ou douce

Hard (retenue)

$\frac{1}{2}\,\mathrm{CE}(\mathrm{cls}, y) + \frac{1}{2}\,\mathrm{CE}(\mathrm{dist}, \mathrm{argmax\ prof.})$

Le CLS reçoit le vrai label, le DIST la décision du professeur. Le label smoothing (0.1) ne touche que les vrais labels.

Soft

$(1-\lambda)\,\mathrm{CE} + \lambda\,\tau^2\,\mathrm{KL}$

La KL compare les distributions adoucies par la température τ. Le facteur τ^2 compense l'affaiblissement des gradients, environ $1/\tau^2$.

83.0 vs 81.8 Dans le papier (DeiT-B, ImageNet), la distillation dure bat la douce. Elle est aussi plus simple : pas d'hyperparamètre τ ni λ.

Classe : DistillationLoss (distillation_type = hard ou soft)

3 · DISTILLATION

Le professeur est un convnet

ResNet-18

Adapté à CIFAR-10 : première convolution 3×3 de pas 1, sans max-pooling. Entraîné sur CIFAR-10 en 32×32.

RegNetY-16GF

Le professeur du papier. Pré-entraîné sur ImageNet, nouvelle tête à 10 classes, ajusté sur CIFAR-10 avec des images agrandies à 224×224.

Un convnet transmet son biais inductif (localité, invariance) que le Transformer n’a pas. Le papier trouve qu’un professeur convnet donne de meilleurs élèves qu’un professeur Transformer. Le professeur est entraîné d’abord, puis figé : poids gelés, mode évaluation, aucun gradient.

Fichier : cifar_teachers.py (ProfesseurFige, AvecAgrandissement)

4 · ENTRAÎNEMENT

Une itération : forward, loss, backward, step

```
cls, dist = student(images)
loss = criterion(cls, dist, labels, teacher_logits)
optimizer.zero_grad(); loss.backward(); optimizer.step()
```

Batch

Groupe d’images traité d’un coup, par exemple 128.

Itération

Une mise à jour des poids, donc un batch.

Epoch

Un passage sur les 50 000 images : 391 itérations avec un batch de 128.

AdamW, learning rate = 5e-4 × batch / 512, warmup linéaire puis décroissance cosinus, scheduler appelé à chaque itération.

Fichiers : train.py, lr_schedule.py

4 · INFÉRENCE

Late fusion : on additionne les probabilités

```
probs = softmax(cls_logits) + softmax(dist_logits)
classe = probs.argmax()
```

Pourquoi le softmax d’abord

Les logits bruts ne sont pas des distributions et n’ont pas la même échelle. Le softmax les met sur une échelle commune avant de les combiner.

Vérification dans le code

Chaque ligne de probs somme à 2. La classe prédite est l’argmax de cette somme.

Méthode : DeiTTinyDistilled.predict